Assignment 1

Individual Design Slice for the CareFlow Course Project

Slice: Clinician Evaluation Queue

Sahid Bangura

EX 601 - Software Engineering

Boston University

October 2026

Contents

Part 1: Scope and Requirements Slice

Part 2: Domain Model

Part 3: Design Model

Part 4: SOLID Decision and ADR

Part 5: AI Step and Critique

Part 6: Reflection and Team Handoff

Project context: CareFlow is a healthcare patient-intake and triage-queue system. This individual slice focuses only on the clinician-facing evaluation workflow.

Part 1: Scope and Requirements Slice

Slice name: Clinician Evaluation Queue

In scope: A clinician can view patients waiting for clinical evaluation in triage-priority order, select a patient to review the patient's triage assessment, and complete the clinical evaluation by recording an outcome and updating that patient's queue status.

Out of scope: Patient intake submission, nurse creation or modification of triage assessments, appointment scheduling, billing, authentication implementation, administrative reporting, and clinic-wide configuration.

Requirements

R1 As a clinician, I can view patients who are waiting for clinical evaluation in priority order, so that patients with greater urgency are considered before lower-priority patients.

R2 As a clinician, I can select a waiting patient and review the patient's current triage assessment, so that I have the relevant information needed before performing the evaluation.

R3 As a clinician, I can record the outcome of an evaluation and update the patient's queue status, so that the system accurately reflects that the patient has been evaluated.

R4 As a clinician, I can refresh the queue and see the current ordering and status of patients, so that I work from the latest available queue state.

Rules / Invariants

- **I1 - supports R1:** Only queue entries with status WAITING may appear in the clinician's waiting queue.
- **I2 - supports R1:** Waiting entries must be ordered by the priority of their current triage assessment; higher-priority patients appear before lower-priority patients.
- **I3 - supports R2:** Only an authorized clinician may view patient and triage-assessment information in this slice.
- **I4 - supports R3:** An evaluation may be completed only for an existing queue entry that is currently waiting.
- **I5 - supports R3:** Completing an evaluation must record the clinician, evaluation time, outcome, and resulting status so the status change remains auditable.
- **I6 - supports R3/R4:** Updating one queue entry must not modify the status or priority of unrelated queue entries.

Part 2: Domain Model

This domain model captures concepts and relationships for the clinician slice. It intentionally avoids Java implementation types. Priority exists on TriageAssessment rather than being duplicated on QueueEntry, so there is one source of truth for the clinical priority used by the queue.

Entities and Key Attributes

Entity	Key attributes (conceptual)
Clinician	clinicianId, name, authorizationRole
Patient	patientId, name

TriageAssessment	priorityLevel, assessmentTime, assessmentNotes
QueueEntry	enteredQueueAt, queueStatus
TriageQueue	queueName, queueDate
Evaluation	evaluatedAt, outcome, resultingStatus

Relationships and Multiplicities

- TriageQueue "1" -> "0..*" QueueEntry : contains
- Patient "1" -> "0..1" QueueEntry : has active entry
- Patient "1" -> "0..*" TriageAssessment : receives
- QueueEntry "1" -> "1" TriageAssessment : uses current assessment
- Clinician "1" -> "0..*" Evaluation : performs
- Patient "1" -> "0..*" Evaluation : receives
- QueueEntry "1" -> "0..1" Evaluation : completed by

Domain Class Diagram

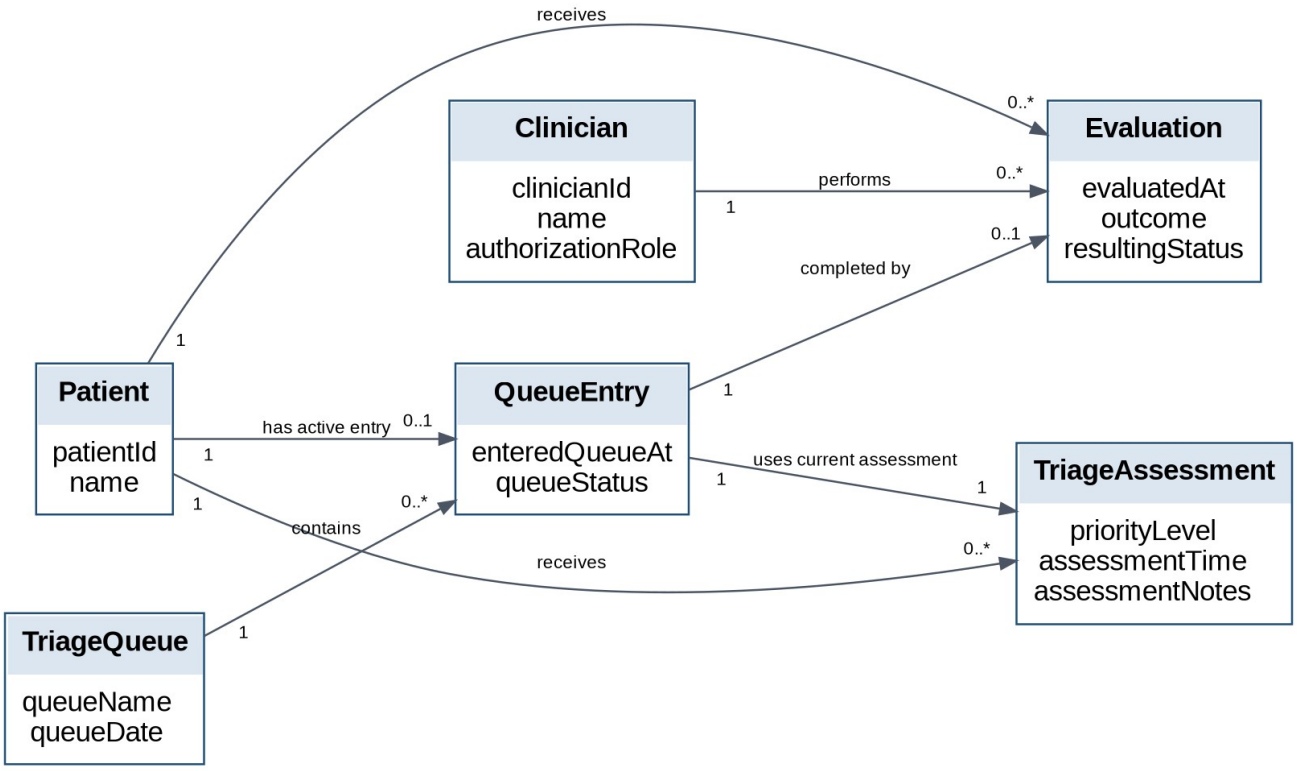


Figure 1. Domain model for the Clinician Evaluation Queue slice.

Part 3: Design Model

The design model assigns system operations to a GRASP Controller, keeps domain rules with the Information Experts that own the relevant data, and isolates persistence behind repository interfaces. The presentation/application layer coordinates the use case; domain classes enforce business rules; the data layer persists state.

Design Class Diagram

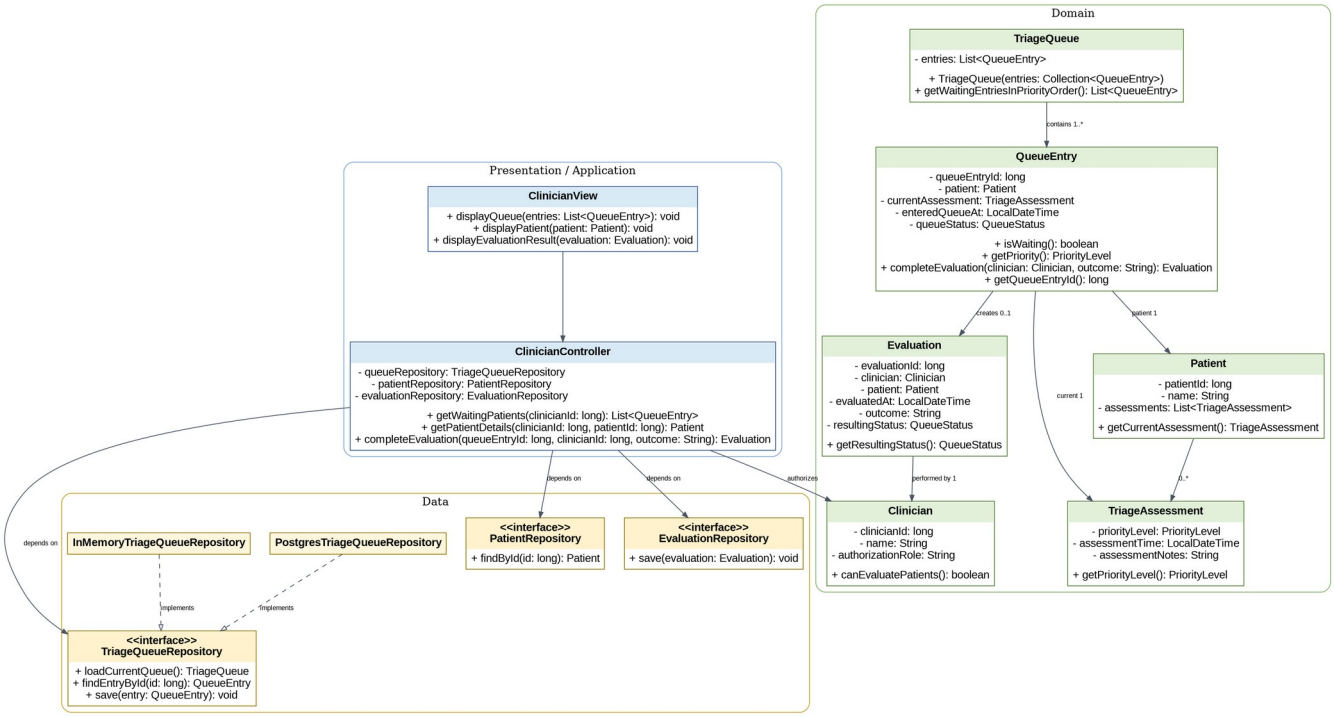


Figure 2. Design class diagram with presentation/application, domain, and data layers.

Responsibility Table

Behavior	Owner class	GRASP	Reasoning
Receive “view queue” and “complete evaluation” system operations	ClinicianController	Controller	First non-UI object that receives and coordinates the use-case request.
Determine whether a queue entry is waiting	QueueEntry	Information Expert	QueueEntry owns queueStatus.
Determine a patient’s triage priority	TriageAssessment	Information Expert	TriageAssessment owns priorityLevel.
Return waiting entries in priority order	TriageQueue	Information Expert	TriageQueue owns the collection being filtered and ordered.
Determine whether a clinician may evaluate patients	Clinician	Information Expert	Clinician owns the authorization role used for this decision.
Provide the current triage assessment	Patient	Information Expert	Patient owns the collection of assessments.
Complete an evaluation and move the entry out of WAITING	QueueEntry	Information Expert / Creator	QueueEntry owns queueStatus and already knows the Patient and current assessment, so it can

			enforce the transition and create the closely related Evaluation.
Persist queue-entry changes	TriageQueueRepository	Low Coupling	Keeps database details out of domain classes and the controller.
Persist completed evaluation	EvaluationRepository	Low Coupling	Separates persistence from evaluation-domain behavior.

Layer Map

Layer	Classes
Presentation / Application	ClinicianView, ClinicianController
Domain	Clinician, Patient, TriageAssessment, QueueEntry, TriageQueue, Evaluation, PriorityLevel, QueueStatus
Data	TriageQueueRepository, PatientRepository, EvaluationRepository, PostgresTriageQueueRepository, InMemoryTriageQueueRepository

Key Design-Level Signatures (Java)

```
public List<QueueEntry> ClinicianController.getWaitingPatients(long clinicianId);

public Patient ClinicianController.getPatientDetails(long clinicianId, long patientId);

public Evaluation ClinicianController.completeEvaluation(
    long queueEntryId, long clinicianId, String outcome);

public boolean Clinician.canEvaluatePatients();

public TriageAssessment Patient.getCurrentAssessment();

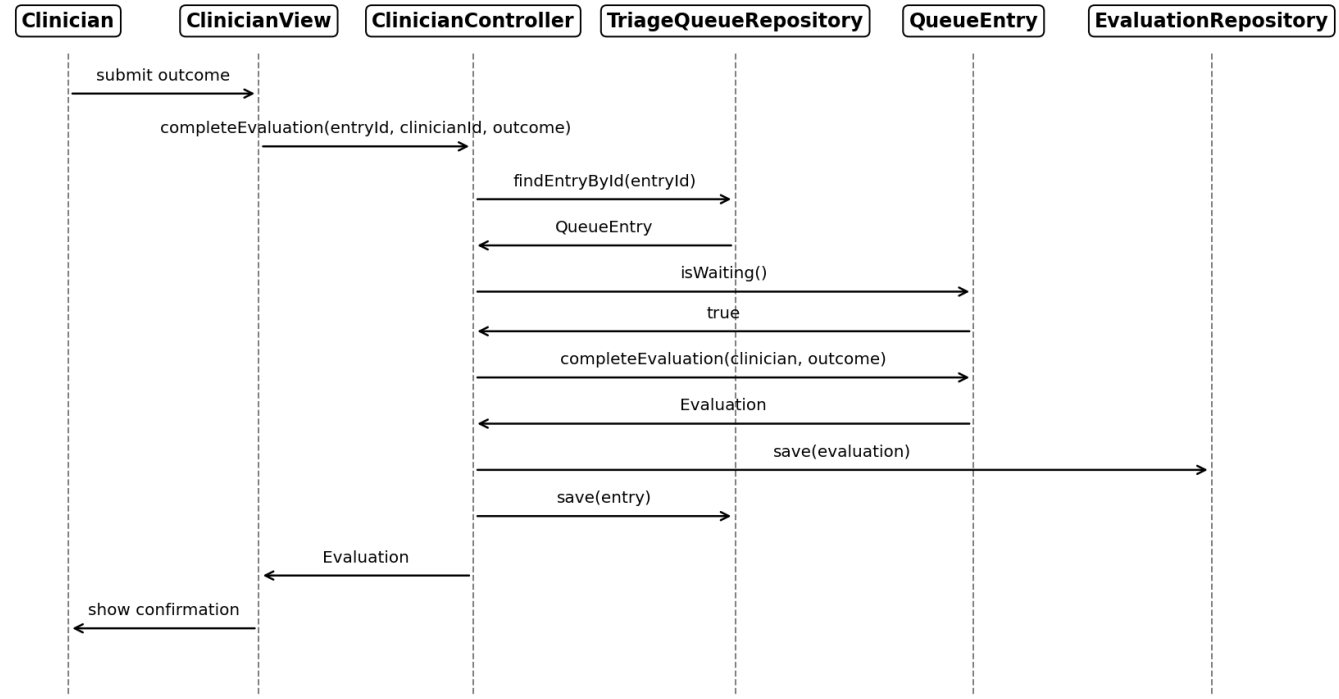
public boolean QueueEntry.isWaiting();
public PriorityLevel QueueEntry.getPriority();
public Evaluation QueueEntry.completeEvaluation(Clinician clinician, String outcome);
public long QueueEntry.getQueueEntryId();

public List<QueueEntry> TriageQueue.getWaitingEntriesInPriorityOrder();

public TriageQueue TriageQueueRepository.loadCurrentQueue();
public QueueEntry TriageQueueRepository.findEntryById(long queueEntryId);
public void TriageQueueRepository.save(QueueEntry entry);

public Patient PatientRepository.findById(long patientId);
public void EvaluationRepository.save(Evaluation evaluation);
```

Sequence Diagram: Complete Patient Evaluation



Flow shown: clinician completes an evaluation; domain object enforces the waiting-state rule before persistence.

Figure 3. Non-obvious evaluation-completion flow. QueueEntry enforces the WAITING-state rule before the controller persists the new Evaluation and updated queue entry.

Part 4: SOLID Decision, with an ADR

Principle in focus: Dependency Inversion Principle (DIP)

Decision path: [X] Applied by design [ ] Refactor [ ] Restraint

The likely change: The same clinician workflow needs two concrete persistence contexts: PostgreSQL-backed storage in production and an isolated in-memory repository during automated tests. The use-case and domain rules should remain unchanged across those contexts.

GRASP question behind it: Low Coupling - can the use-case logic avoid depending on SQL and database-specific details?

Java Structure

```
public interface TriageQueueRepository {
   TriageQueue loadCurrentQueue();
   QueueEntry findEntryById(long queueEntryId);
   void save(QueueEntry entry);
}

public final class PostgresTriageQueueRepository
      implements TriageQueueRepository {

   @Override
   public TriageQueue loadCurrentQueue() {
      // PostgreSQL query and row-mapping logic
```

```
        throw new UnsupportedOperationException("database code omitted");
    }

    @Override
    public QueueEntry findEntryById(long queueEntryId) {
        // PostgreSQL lookup
        throw new UnsupportedOperationException("database code omitted");
    }

    @Override
    public void save(QueueEntry entry) {
        // PostgreSQL update
    }
}

public final class InMemoryTriageQueueRepository
        implements TriageQueueRepository {

    private final Map<Long, QueueEntry> entries = new HashMap<>();

    @Override
    public TriageQueue loadCurrentQueue() {
        return new TriageQueue(entries.values());
    }

    @Override
    public QueueEntry findEntryById(long queueEntryId) {
        return entries.get(queueEntryId);
    }

    @Override
    public void save(QueueEntry entry) {
        entries.put(entry.getQueueEntryId(), entry);
    }
}

public final class ClinicianController {
    private final TriageQueueRepository queueRepository;

    public ClinicianController(TriageQueueRepository queueRepository) {
        this.queueRepository = queueRepository;
    }
}
```

ADR-001: Keep the Clinician Workflow Independent of Persistence Details

Status: Accepted

Date: 2026-10-04

Context

The clinician slice loads the current queue, retrieves a queue entry, changes that entry after evaluation, and saves the completed evaluation. Production requires database-backed persistence, while automated

tests need to exercise the same clinician workflow without requiring a live database. If ClinicianController contained SQL or instantiated a concrete database repository directly, changes to persistence and test setup would spread into use-case code.

Decision

Apply the Dependency Inversion Principle. ClinicianController depends on the TriageQueueRepository abstraction. PostgresTriageQueueRepository provides the production implementation, while InMemoryTriageQueueRepository provides a lightweight implementation for isolated automated tests. Domain classes remain unaware of persistence details.

Consequences

- Positive: The clinician workflow and domain rules can be tested without a live PostgreSQL database.
- Positive: SQL and row-mapping changes remain localized to the data layer instead of forcing changes to the controller or domain classes.
- Negative: The design adds an interface and multiple classes that developers must navigate.
- Negative: Repository abstractions must be kept small and domain-focused; otherwise they can become generic data-access layers that add indirection without protecting a real change.

Part 5: The AI Step

Prompt Used

> Here is part of my CareFlow clinician design. A ClinicianController retrieves queue entries using a TriageQueueRepository, allows a clinician to view patients in priority order, and records an Evaluation after a patient is seen. QueueEntry owns its queue status and TriageQueue owns priority ordering. Make this design more SOLID and extensible.

Assistant Output (relevant portion)

The assistant proposed the following changes:

- Create a QueueOrderingStrategy interface with PriorityQueueOrderingStrategy and FifoQueueOrderingStrategy implementations.
- Create an EvaluationCompletionService interface so evaluation completion can have multiple implementations.
- Create a ClinicianAuthorizationService interface instead of letting Clinician determine authorization.
- Keep repository interfaces so the use-case logic is independent of persistence details.
- Add QueueEntryFactory and EvaluationFactory classes so object construction can be replaced independently.

Critique (150-250 words)

The AI applied several SOLID ideas, especially Dependency Inversion, Open/Closed, and Single Responsibility. I would keep TriageQueueRepository because it has two concrete needs in this design: a PostgreSQL implementation for production and an in-memory implementation for isolated automated testing. That abstraction protects a real boundary and lets the clinician workflow remain unchanged across both contexts. I would push back on most of the additional abstractions. QueueOrderingStrategy is not justified by the current slice because CareFlow has one explicit business rule: waiting patients are ordered by triage priority. There is no current FIFO or interchangeable ordering requirement. Adding multiple ordering strategies would therefore model hypothetical behavior rather than a likely change. I would also reject EvaluationCompletionService. QueueEntry already owns queueStatus and is the

Information Expert for the WAITING-to-evaluated transition, while ClinicianController coordinates the use case. Another service would mostly redirect calls. The same concern applies to QueueEntryFactory and EvaluationFactory; the objects are straightforward to create and there is no second construction policy. I would therefore accept the repository abstraction but reject the speculative strategy, service, and factory layers. The useful test is whether an abstraction protects a concrete variation that actually exists or is likely, rather than merely making the design look extensible.

Part 6: Reflection and Team Handoff

Reflection (150-250 words)

The SOLID principle most at work in my clinician slice was Dependency Inversion. The clinician workflow must load and update queue data, but the controller and domain objects should not need to know whether that data comes from PostgreSQL or an in-memory test repository. Depending on a repository interface keeps persistence mechanics outside the use-case logic and lets the same domain behavior run in both contexts. I know this decision is warranted rather than over-engineered because the two implementations represent concrete needs now: production persistence and isolated automated testing. The abstraction therefore protects a real variation. The AI exercise also clarified where the line should stop. Interfaces such as QueueOrderingStrategy, EvaluationCompletionService, and object factories would not currently protect against an actual requirement. CareFlow has one defined triage-ordering rule, and QueueEntry already owns the status transition after evaluation. Adding more layers there would increase the number of files and dependencies without making a likely change easier. The main practice I would carry forward is to name the specific variation before introducing an interface. If I can point to a real second implementation or a near-term change, abstraction may earn its keep. If I cannot, I should keep the design simple and revisit it when the second case appears.

Team Handoff

Slice I covered: Clinician Evaluation Queue.

Strongest piece to feed Checkpoint 1: The design model, because it assigns queue ordering, status-transition, evaluation-creation, authorization, and persistence responsibilities to explicit owners and gives the team a concrete three-layer structure to integrate with the other CareFlow slices.

Submission Checklist

- Part 1 has a stated in-scope and out-of-scope boundary, four user-story requirements, and six traceable invariants.
- Part 2 has six domain entities, conceptual attributes, relationships, multiplicities, and a domain UML diagram.
- Part 3 has a design UML diagram with visibility and types, a GRASP responsibility table, a three-layer map, Java signatures, and a sequence diagram.
- Part 4 names one SOLID principle, uses Applied by design, states a concrete variation, shows one interface with two implementations, and includes an ADR with positive and negative consequences.
- Part 5 includes the prompt, the assistant output, and a 150-250 word critique identifying principles, over-abstraction, and accept/push-back decisions.
- Part 6 includes a 150-250 word reflection and a specific 1-2 sentence team handoff.